

Payment Failure Intelligence Platform

Production Ready Agentic AI + RAG Solution

1. Project Overview

Project Name

Payment Failure Intelligence Platform

Domain

ERCOT Generation Interconnection Request (GINR) Operations

Objective

Build an AI-powered conversational platform that enables operations teams to investigate payment failures, identify root causes, analyze trends, and retrieve historical resolutions using natural language.

2. Business Problem

ERCOT receives thousands of payment transactions related to:

- Full Interconnection Study (FIS)
- Scoping Study (SS)
- Change Requests
- Analysis Fees
- Interconnection Requests

Operations teams manually review payment reports generated daily.

Problems:

Manual Investigation

Teams search through hundreds of:

- CSV Reports
- HTML Reports
- Excel Reports

- Email Notifications

to answer simple questions.

Example:

Why did GINR-12345 fail?

No Historical Intelligence

Finding similar failures requires manual effort.

Slow Resolution

Support engineers spend hours identifying:

- Root causes
- Previous resolutions
- Payment trends

Limited Visibility

Management lacks insights into:

- Failure trends
- Most common issues
- Operational bottlenecks

3. Business Objectives

The platform should:

1. Read all historical payment reports.
 2. Store operational knowledge.
 3. Answer payment-related questions.
 4. Identify root causes.
 5. Recommend corrective actions.
 6. Generate executive summaries.
 7. Analyze payment trends.
-

4. Proposed Solution

Build a RAG-based AI platform using:

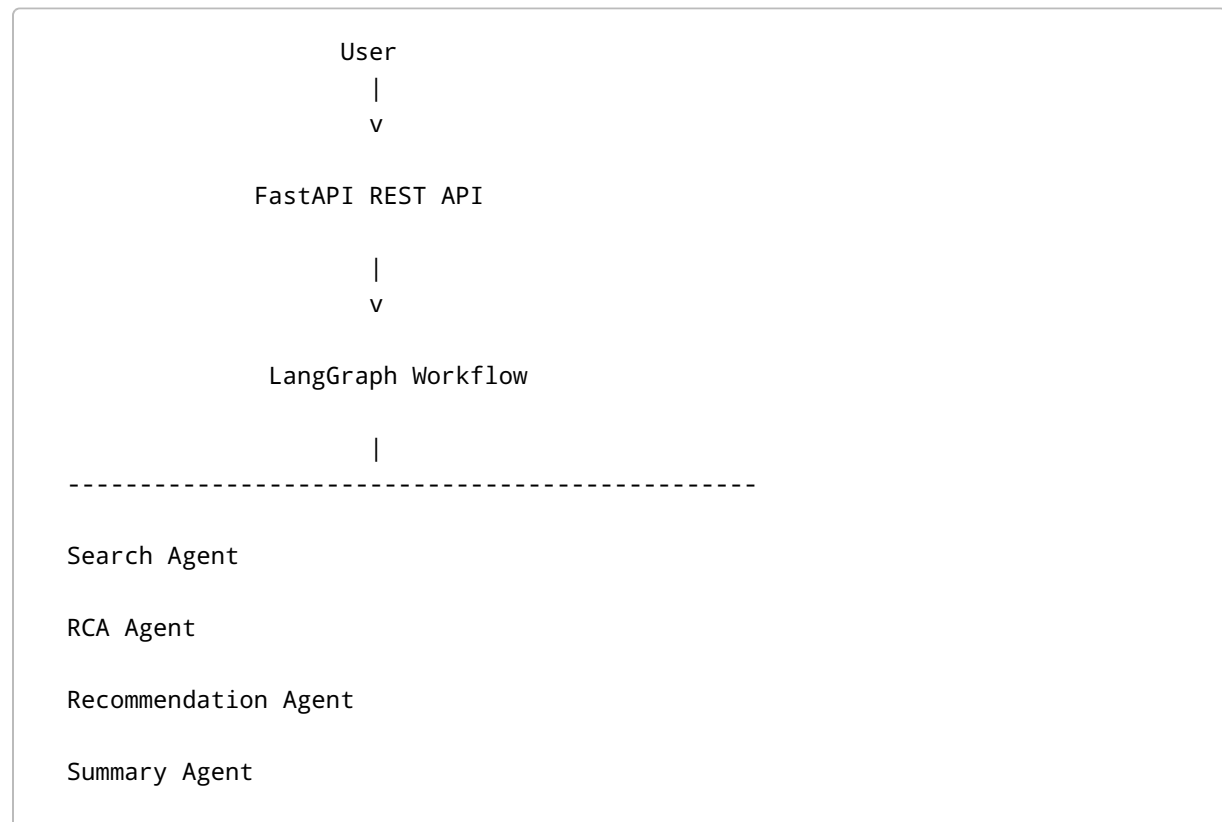
- LangGraph
- LangChain
- Ollama
- ChromaDB
- FastAPI

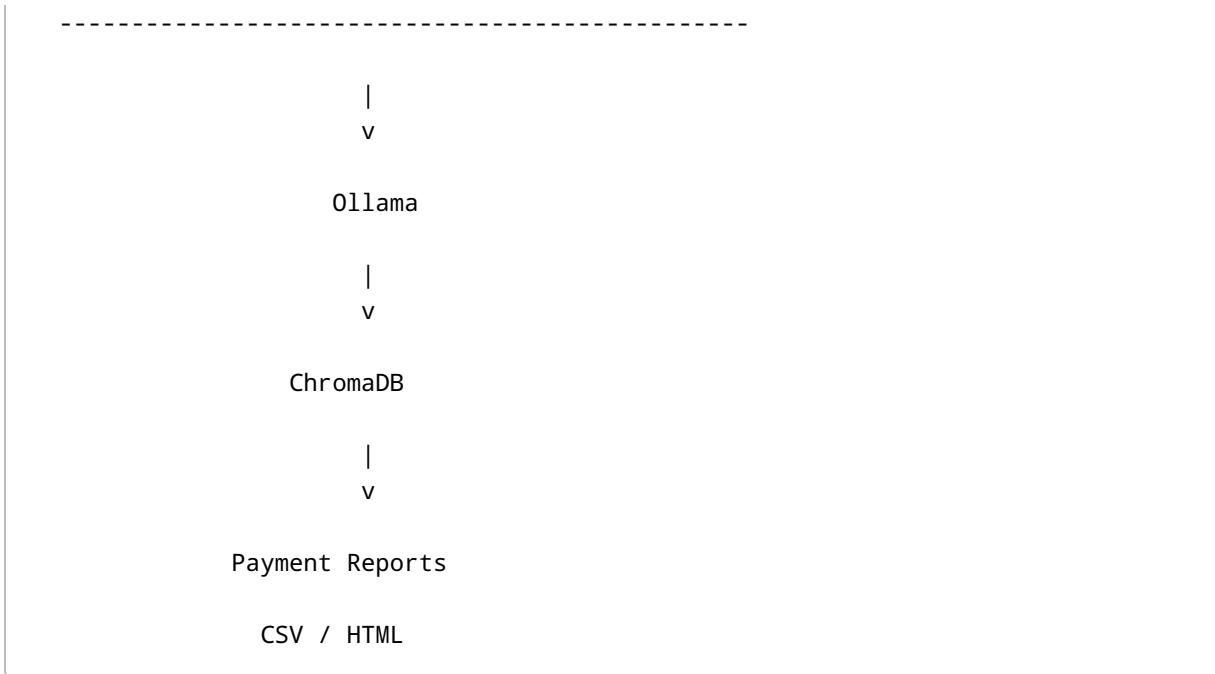
The solution transforms historical payment reports into searchable knowledge.

Users can ask:

- Why did GINR-12345 fail?
- Show all duplicate payments.
- What are the top payment failures in the last three months?
- Show payment trends for 2025.
- Which failures are increasing?

5. High-Level Architecture





6. Technology Stack

Backend

Python 3.12

FastAPI

Agent Framework

LangGraph

LangChain

LLM

Ollama

Llama3

Embeddings

nomic-embed-text

Vector Database

ChromaDB

Data Sources

CSV Reports

HTML Reports

Oracle (Future)

Deployment

Docker

Kubernetes

Monitoring

Prometheus

Grafana

7. System Components

Component 1

Payment Monitor

Generates daily reports.

Examples:

2025-01-payment-report.csv

2025-01-summary.html

2025-02-payment-report.csv

2025-02-summary.html

Component 2

Ingestion Pipeline

Responsibilities:

- Read reports
 - Extract records
 - Create documents
 - Generate embeddings
-

Component 3

Embedding Service

Model:

nomic-embed-text

Converts text into vectors.

Component 4

Vector Database

ChromaDB

Stores:

- Failure information
 - Severity
 - Resolution
 - Project information
-

Component 5

LangGraph Agents

Responsible for intelligent processing.

Component 6

FastAPI

Provides REST APIs.

8. Data Flow

Step 1

Payment Monitor generates reports.

Example:

January Reports

February Reports

March Reports

Step 2

Ingestion Job Executes

Command:

```
python -m rag.ingest
```

Step 3

CSV and HTML files loaded.

Step 4

Documents created.

Example:

Project ID: GINR123

Failure Type: FIS Fee Missing

Severity: Critical

Resolution: Contact Customer

Step 5

Embeddings generated.

Model:

nomic-embed-text

Step 6

Vectors stored in ChromaDB.

Step 7

User asks question.

Example:

Why did GINR123 fail?

Step 8

Retriever searches vector database.

Step 9

Relevant incidents retrieved.

Step 10

LangGraph workflow executes.

Step 11

Answer generated.

9. LangGraph Architecture

Why LangGraph?

Traditional applications execute sequentially.

LangGraph enables:

- Multi-Agent Systems
 - State Management
 - Conditional Routing
 - Complex AI Workflows
-

10. Agent Workflow

User Question

↓

Search Agent

↓

Root Cause Agent

↓

Recommendation Agent

↓

Summary Agent

↓

Final Response

11. Agent Responsibilities

Search Agent

Purpose:

Retrieve relevant incidents from ChromaDB.

Input:

Why did GINR123 fail?

Output:

Relevant payment records.

Root Cause Agent

Purpose:

Analyze retrieved data.

Output:

FIS Fee Missing

Recommendation Agent

Purpose:

Suggest corrective actions.

Output:

Contact customer.

Summary Agent

Purpose:

Generate final response.

Output:

GINR123 failed because FIS Fee was not received.

12. State Flow

Initial State

{ question }

↓

Search Agent

{ question context }

↓

RCA Agent

{ question context root_cause }

↓

Recommendation Agent

{ question context root_cause recommendation }

↓

Summary Agent

{ question context root_cause recommendation answer }

13. Project Structure

payment-ai-platform

src

api

main.py

routes.py

agents

search_agent.py

rca_agent.py

recommendation_agent.py

summary_agent.py

rag

ingest.py

retriever.py

embeddings.py

graph.py

data

2025

2026

chroma_db

requirements.txt

14. API Design

Health Endpoint

GET /health

Response

```
{ "status": "UP" }
```

Ask Endpoint

POST /ask

Request

```
{ "question": "Why did GINR123 fail?" }
```

Response

```
{ "answer": "FIS Fee Missing" }
```

15. Runbook

Prerequisites

Install:

Python 3.12

Ollama

Git

Clone Repository

```
git clone payment-ai-platform
```

```
cd payment-ai-platform
```

Create Virtual Environment

```
python -m venv .venv
```

Windows

```
.venv\Scripts\activate
```

Linux

```
source .venv/bin/activate
```

Install Dependencies

```
pip install -r requirements.txt
```

Start Ollama

```
ollama serve
```

Pull Models

```
ollama pull llama3
```

```
ollama pull nomic-embed-text
```

Verify Models

```
ollama list
```

Load Reports into ChromaDB

```
python -m rag.ingest
```

Expected Output

Found 24 files

Loaded 3500 records

ChromaDB Created Successfully

Start FastAPI

```
uvicorn api.main:app --reload
```

Health Check

<http://localhost:8000/health>

Swagger

<http://localhost:8000/docs>

Ask Questions

POST

/ask

Example:

```
{ "question": "Why did GINR123 fail?" }
```

16. Sample Questions

Operational

Why did GINR123 fail?

Show duplicate payments.

Show payment synchronization failures.

Show all critical failures.

Management

Show trends for last three months.

Which failure type is increasing?

What are top payment issues?

Show monthly failure distribution.

Executive

Generate quarterly summary.

Show risk areas.

Recommend operational improvements.

17. Production Deployment

Docker Container

↓

Kubernetes Deployment

↓

Load Balancer

↓

FastAPI Pods

↓

ChromaDB

↓

Ollama

↓

Persistent Storage

18. Monitoring

Prometheus Metrics

API Requests

Latency

Token Usage

Embedding Time

Grafana Dashboards

API Health

Agent Performance

Question Volume

Failure Trends

19. Resume Description

Designed and developed an AI-powered Payment Failure Intelligence Platform using LangGraph, LangChain, Ollama, ChromaDB, and FastAPI. Implemented Retrieval-Augmented Generation (RAG) pipelines to ingest historical payment reports, perform semantic search, identify root causes, generate recommendations, and provide conversational analytics for ERCOT payment operations. Built a multi-agent architecture with Search, Root Cause Analysis, Recommendation, and Summary agents, enabling faster incident investigation and improved operational visibility.

20. Interview Explanation (5 Minutes)

The business problem was that ERCOT operations teams manually investigated payment failures using CSV, HTML, and Excel reports. As the volume of reports increased, it became difficult to identify root causes, find similar incidents, and generate management insights.

To solve this, I designed an AI-powered Payment Failure Intelligence Platform using LangGraph, LangChain, Ollama, ChromaDB, and FastAPI. Historical payment reports are ingested through a RAG pipeline, converted into embeddings using the nomic-embed-text model, and stored in ChromaDB. When a user asks a

question, a LangGraph workflow orchestrates multiple agents including Search, Root Cause Analysis, Recommendation, and Summary agents. These agents retrieve relevant historical incidents, analyze the issue, recommend actions, and generate human-readable responses.

The platform enables conversational access to operational knowledge, reduces manual investigation effort, improves issue resolution time, and provides management-level insights into payment failure trends and operational risks.